

Doc. no.:	P0849R0
Date:	2017-11-02
Audience:	EWG, LEWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

auto(x): *DECAY_COPY* in the language

Introduction

This paper proposes `auto(x)` and `auto{x}` for transforming `x` into a prvalue with the same value as-if passed as a function argument by value. When users asked for this functionality, we claimed that the *DECAY_COPY* notion in the standard serves such purpose, but it is for exposition only.

Motivation

A generic way to obtain copies of an object in C++ is `auto a = x;` but this gives lvalue. It is often convenient to obtain a copy as a prvalue. For example:

1. When using fluent interface for mutation,

```
parser.add_option(...).add_option(...);
```

You may want to make changes on a fork. Instead of writing

```
auto subparser = parser;  
subparser.add_option(...);
```

, with this proposal, you can write

```
auto subparser = auto(parser).add_option(...);
```

2. When you are about to run into the trouble of mutating an object while using its subobject for reference,

```
// paths = { /home/zy
//           /home/zy/devel
//           /home/zy/temp }
replace_prefixes(paths, paths[0], ".");
// paths = { .
//           /home/zy/devel
//           /home/zy/temp }
```

, instead of writing

```
auto p = paths[0];
replace_prefixes(paths, p, ".");
// paths = { .
//           ./devel
//           ./temp }
```

, with this proposal, you can write

```
replace_prefixes(paths, auto(paths[0]), ".");
```

Discussion

Replacing the `char` in `char('a')` with `auto`, we obtain `auto('a')`, which is a function-style cast. Such a formula also supports *injected-class-names* and class template argument deduction in C++17. Introducing `auto(x)` and `auto{x}` significantly improves the language consistency. The following table shows how:

variable definition	function-style cast	new expression
<code>auto v(x);</code>	<code>auto(x)</code>	<code>new auto(x)</code>
<code>auto v{x};</code>	<code>auto{x}</code>	<code>new auto{x}</code>
<code>ClassTemplate v(x);</code>	<code>ClassTemplate(x)</code>	<code>new ClassTemplate(x)</code>
<code>ClassTemplate v{x};</code>	<code>ClassTemplate{x}</code>	<code>new ClassTemplate{x}</code>

****** *The type of `x` is a specialization of `ClassTemplate`.*

With this proposal, all the cells in the table copy construct form `x` (given CTAD's default behavior) to obtain lvalues, prvalues, and pointers to objects, categorized by their columns. That is why this paper does not recommend defining such a facility in the library^[1].

Introducing `auto(x)` into the language even improves the library consistency:

type function style	expression style
<code>void_t<decltype(expr)></code>	<code>decltype(void(expr))</code>
<code>decay_t<decltype(expr)></code>	<code>decltype(auto(expr))</code>

Should we also allow `decltype(auto){x}` ? Maybe not. It is a no-op, different from `new decltype(auto){x}` .

Implementation

Try it out: <https://godbolt.miator.net/g/uw8nKr> (<https://godbolt.miator.net/g/uw8nKr>)

Wording

The wording is relative to N4700.

Modify 8.2.3 [expr.type.conv]/1 as indicated:

A *simple-type-specifier* (10.1.7.2) or *typename-specifier* (17.7) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (16.3.1.8) for the remainder of this section. Otherwise, if the type is `auto`, it is replaced by the type deduced for the variable `x` in the invented declaration (10.1.7.4):

```
auto x init;
```

, where *init* is the initializer.

Modify 10.1.7.4 [dcl.spec.auto]/4 as indicated:

A placeholder type can also be used in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (8.3.4) and as a *decl-specifier* of the *parameter-declaration's decl-specifier-seq* in a *template-parameter* (17.1). The `auto` *type-specifier* can also be used as the *simple-type-specifier* in an explicit type conversion (functional notation) (8.2.3).

Acknowledgments

Thank Arthur O'Dwyer and Billy O'Neal for criticizing an early draft of this paper.

References

1. Krügler, Daniel. P0758R0 *Implicit conversion traits and utility functions*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0758r0.html>) ↩